

TOPA 3급 온라인

02 빅데이터와 파이썬



목차

—

AI와 빅데이터의 개념과
데이터 분석의 정의 및
방법론, pandas의 주요
문법을 알아봅니다.

- 01 **AI와 빅데이터**
- 02 **빅데이터의 특징**
- 03 **데이터 분석 계획**
- 04 **빅데이터 활용 기술**



목차

—

AI와 빅데이터의 개념과
데이터 분석의 정의 및
방법론, pandas의 주요
문법을 알아봅니다.

05 데이터 변환

06 Pandas

07 Pandas 문법 (1)

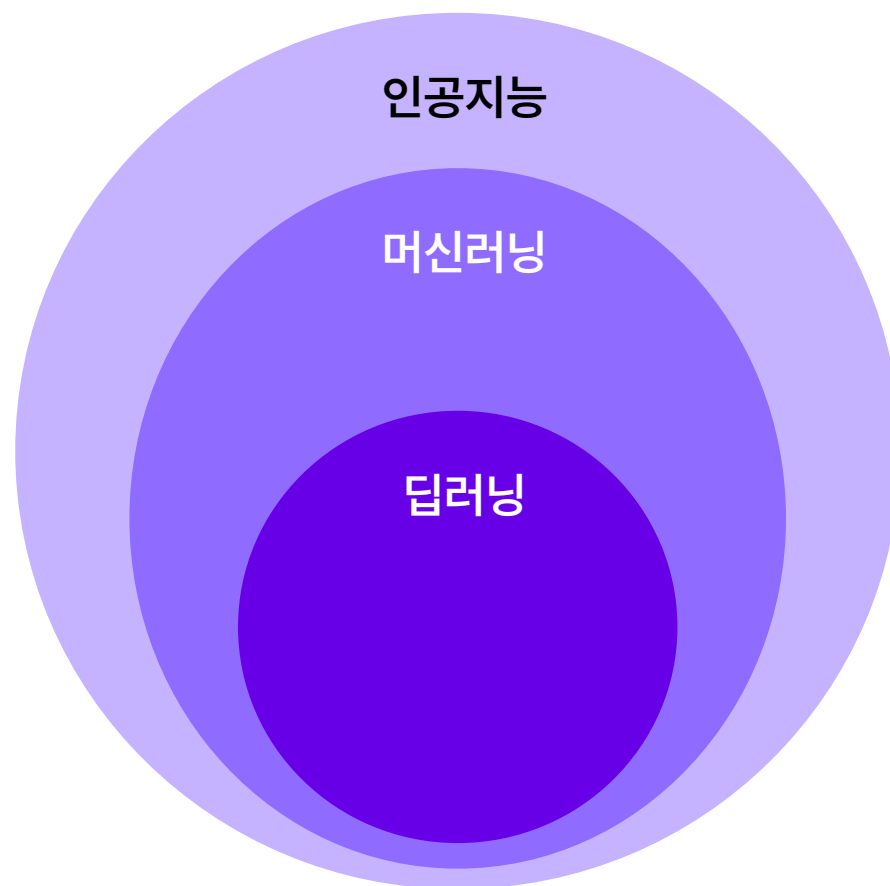
08 Pandas 문법 (2)

09 데이터 집계 및 요약

01

AI와 빅데이터

☑ 인공지능? 머신러닝? 딥러닝?



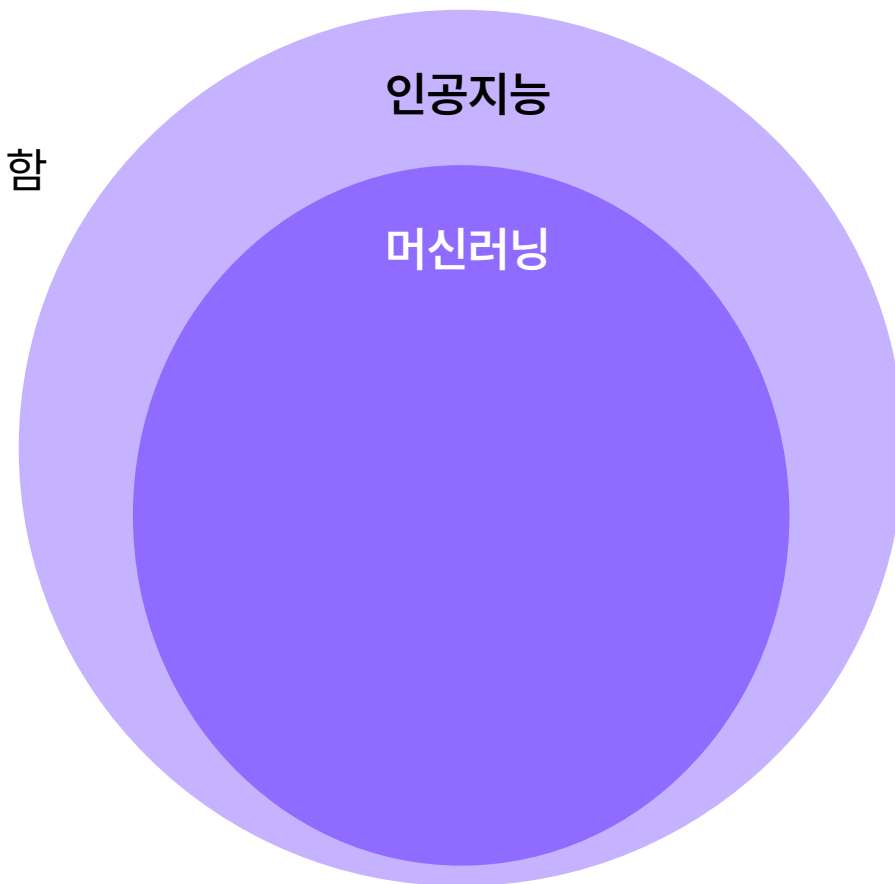
✓ 인공지능 (Artificial Intelligence)

- 컴퓨터 시스템이 인간과 유사한 지능을 가지도록 하는 기술
- 학습, 추론, 인식, 계획, 지각, 개체 조작 능력 문제 등을 다룸



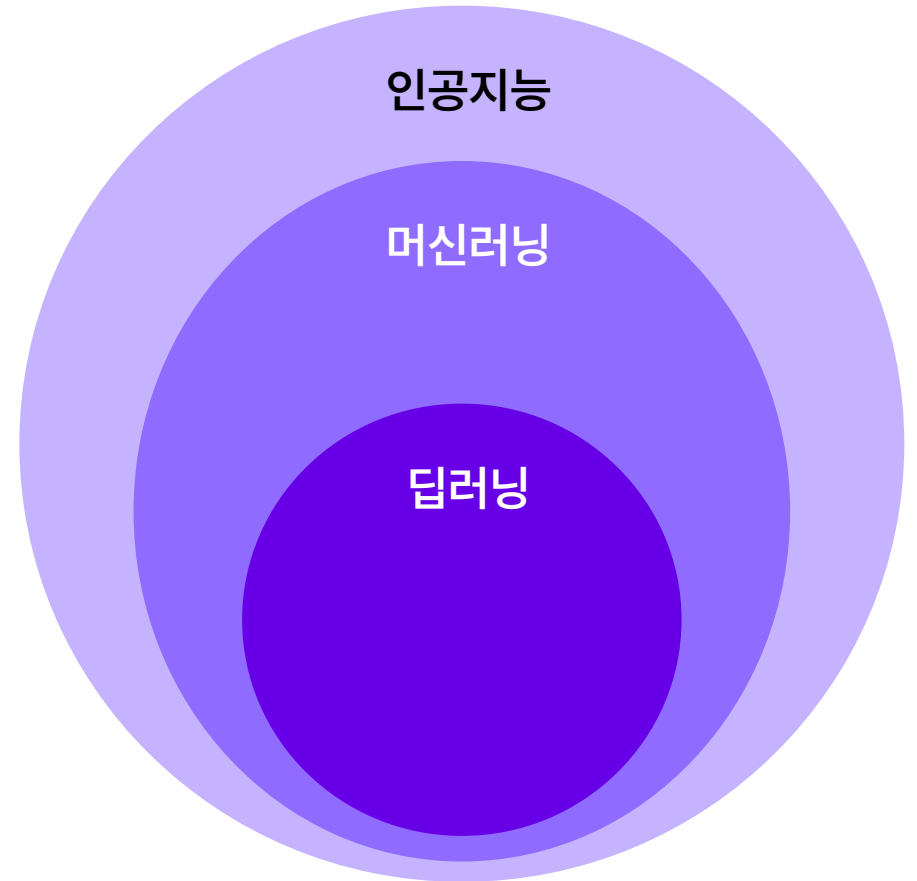
☑ 머신러닝 (Machine Learning)

- 인공지능의 한 분야
- 데이터를 통해 컴퓨터가 스스로 학습하고 의사결정을 내리는 능력을 갖게 함
- 지도학습 (Supervised Learning)과 비지도학습 (Unsupervised Learning)으로 나뉨
 - 지도 학습: 정답이 있는 데이터로 학습
ex) label이 있는 강아지와 고양이 사진 분류
 - 비지도 학습: 정답이 없는 데이터로 학습
ex) label이 없는 데이터 클러스터링



✓ 딥러닝 (Deep Learning)

- 사람의 신경망을 모사한 인공 신경망을 기반하는 머신러닝의 한 종류
- 기존에 풀기 어려웠던 복잡한 문제를 해결 가능



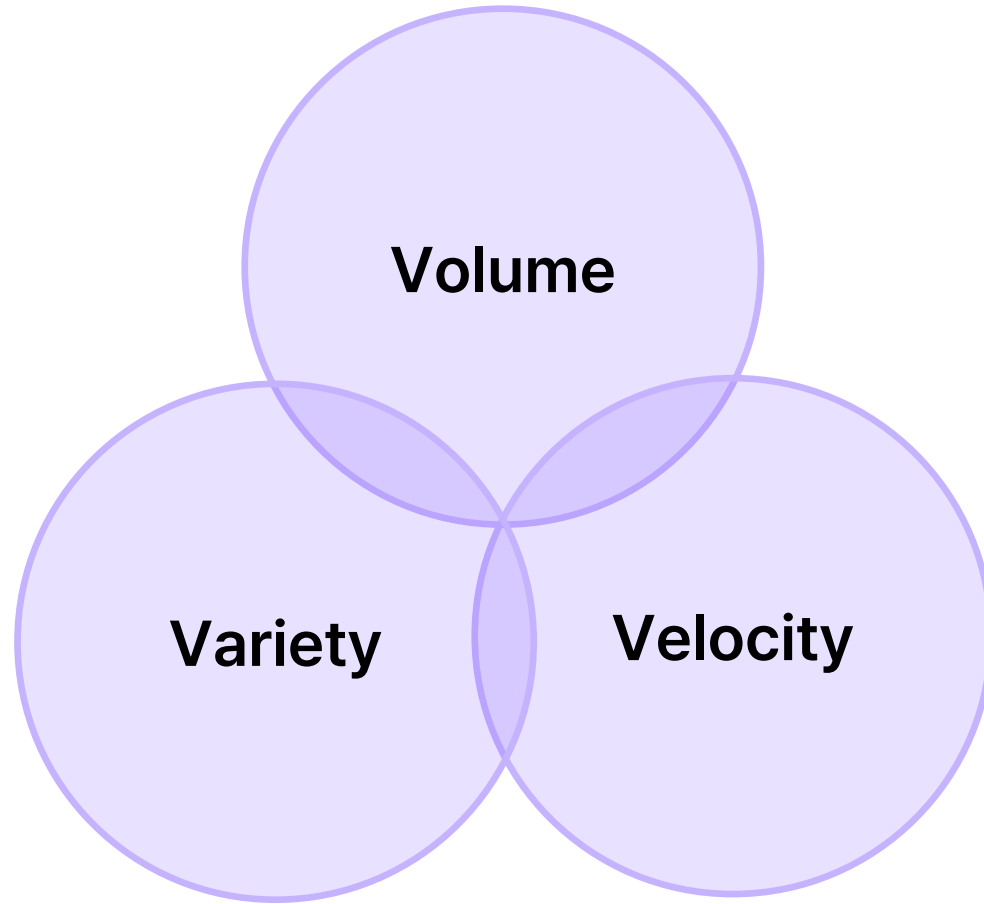
✓ 빅데이터 (Big Data)

- 통상적으로 사용되는 데이터 수집, 관리 및 처리 소프트웨어의 수용 한계를 넘어서는 크기의 데이터
- 데이터의 양이 방대하고 지속적으로 증가하여 전통적인 방법으로는 분석할 수 없는 데이터

02

빅데이터의 특징

✓ 빅데이터 3V



④ 빅데이터 3V

- 1) Volume (규모): 테라바이트 수준의 규모를 갖는 데이터
- 2) Variety (다양성): 정형/비정형의 다양한 형태의 데이터
- 3) Velocity (속도): 실시간 생산, 처리, 유통되는 매우 빠른 데이터

☑ 빅데이터 4V

- 1) Volume (규모): 테라바이트 수준의 규모를 갖는 데이터
- 2) Variety (다양성): 정형/비정형의 다양한 형태의 데이터
- 3) Velocity (속도): 실시간 생산, 처리, 유통되는 매우 빠른 데이터
- 4) Value (가치): 함축된 가치를 찾아내는 데이터

ROI 관점

투자비용
(Investment)

비즈니스 효과
(Return)



ROI

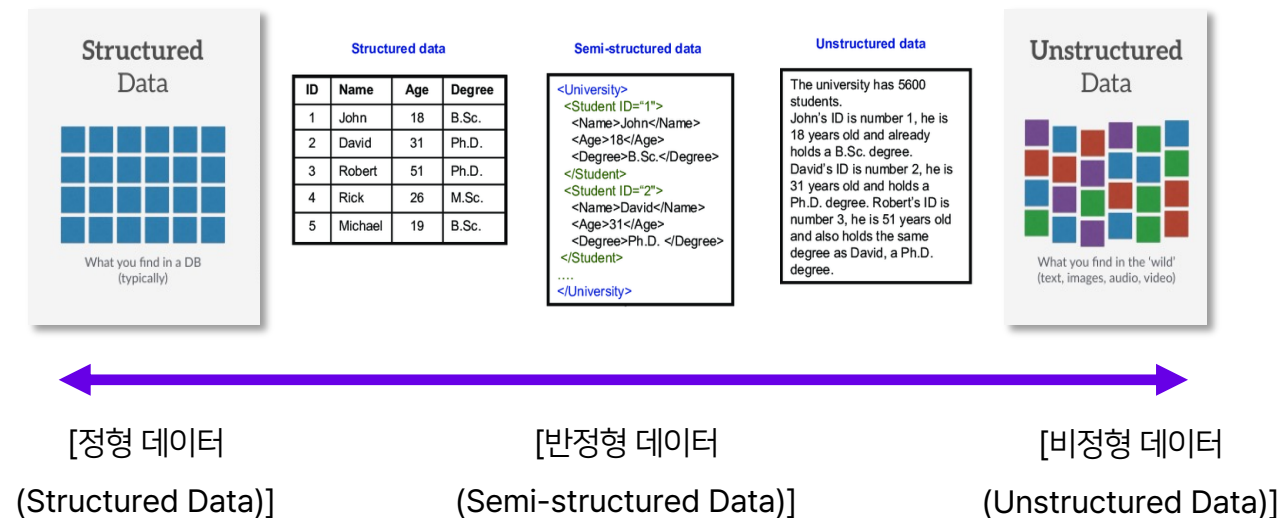
Return On Investment의 약자로 투자 이익률을 의미함.
투자액에 대해 얼마나 이익이 올랐는지를 나타내는 지표

④ 빅데이터 5V

- 1) Volume (규모): 테라바이트 수준의 규모를 갖는 데이터
- 2) Variety (다양성): 정형/비정형의 다양한 형태의 데이터
- 3) Velocity (속도): 실시간 생산, 처리, 유통되는 매우 빠른 데이터
- 4) Value (가치): 함축된 가치를 찾아내는 데이터
- 5) Veracity (진실성): 신뢰성과 타당성이 존재하는 데이터

☑ 데이터의 종류

- 1) 정형 데이터: 구조화된 데이터. 열과 행으로 이루어진 테이블 형식
- 2) 비정형 데이터: 구조화되지 않은 데이터. 비디오, 오디오, 텍스트 등의 특정 구조를 갖지 않은 데이터
- 3) 반정형 데이터: 정형 데이터는 아니지만 어느정도 구조화가 되어있는 데이터. XML, HTML, JSON 형식

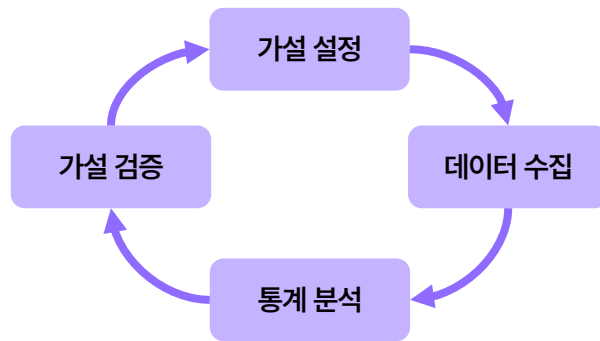


03

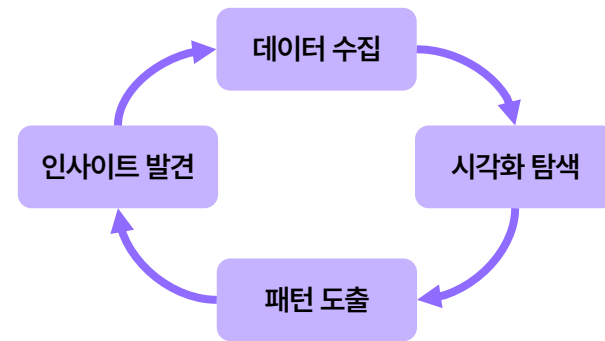
데이터 분석 계획

☑ 데이터 분석

- 특정 타겟의 패턴 또는 트렌드와 같은 유용한 인사이트를 발견하기 위해 데이터를 수집·정리·조작·분석하는 전 과정
- 통계 분석, 머신러닝, 데이터 마이닝 등의 방법을 사용하여 분석



확증적 데이터 분석 (CDA)



탐색적 데이터 분석 (EDA)

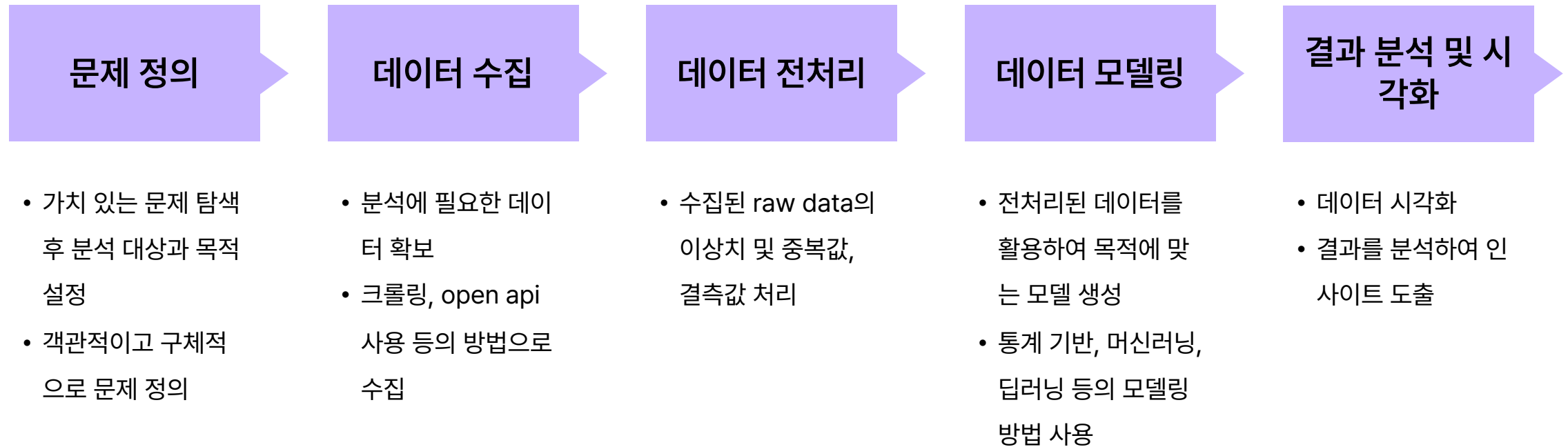
✓ 데이터 분석 특징

- 데이터를 기반으로 인사이트를 도출하는 것이므로 양질의 데이터를 얼마나 많이 확보하느냐가 중요함
- 데이터와 함께 비즈니스의 목표 및 핵심 가치 등을 함께 고려하여 분석 해야함

☑ 데이터 분석의 장점

- 효율적인 의사 결정에 도움
- 타겟 이해도와 만족도를 높임
- 마케팅 최적화를 도움
- 업무 효율성을 높임
- 예측 가능성을 높여 비즈니스 리스크 관리를 강화함

☑ 데이터 분석 순서



04

빅데이터 활용 기술

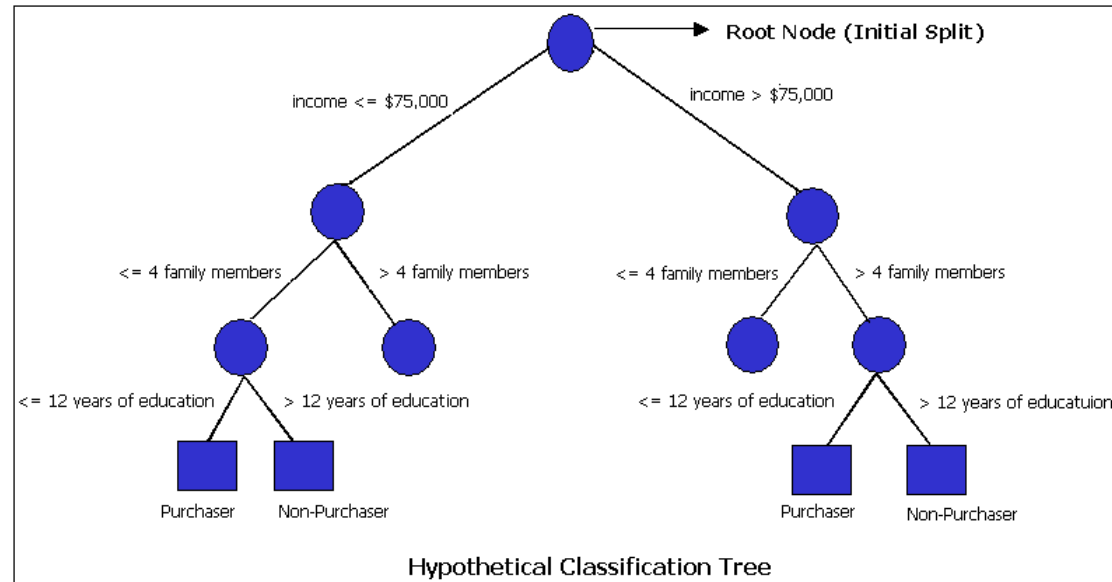
✓ 연관 규칙 학습 (Association rule learning)

- 빅데이터 속 변수들 간의 관계를 발견하기 위한 기법
 - 슈퍼마켓 판매 데이터에서 찾아진 (감자, 양파) → 햄버거 규칙



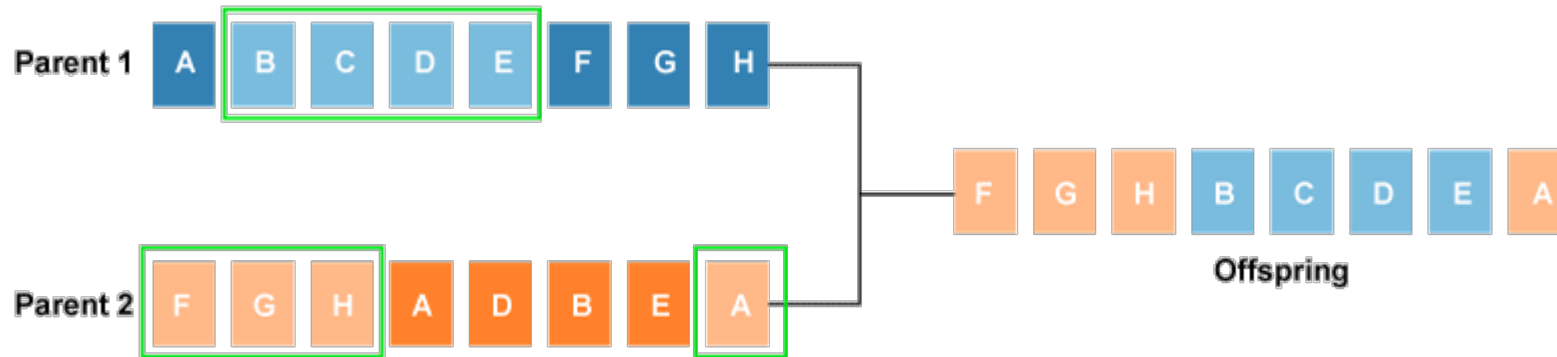
④ 유형 분석 (Classification tree analysis)

- 통계적 분류를 활용하여 사건이 속하게 될 범주를 찾아내는 기법
 - 이 사용자는 어떠한 특성의 집단에 속하는가?
 - 문서 분류, 조직 분류 등에 활용



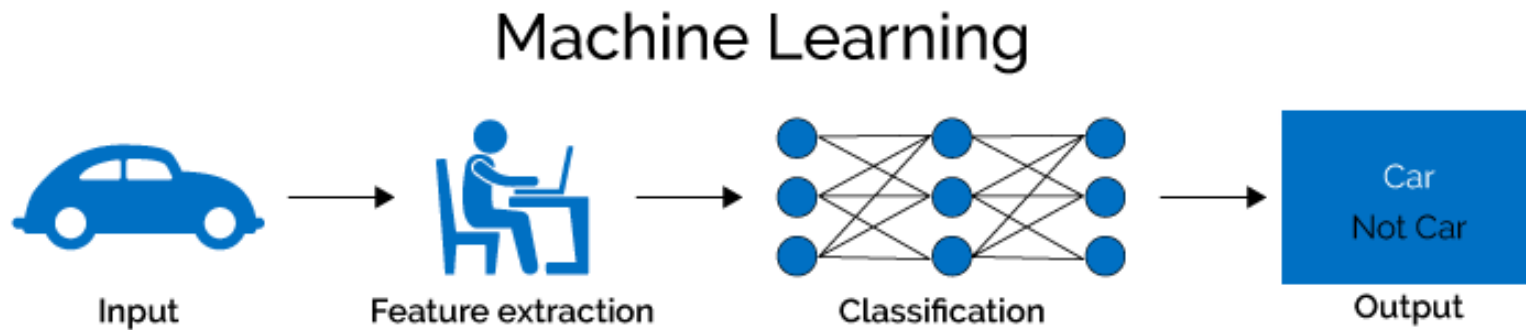
☑ 유전 알고리즘 (Genetic algorithms)

- 최적화가 필요한 문제의 해결책을 자연 선택, 돌연변이 등과 같은 메커니즘을 통해 점진적으로 진화시켜 나가는 기법
 - 응급실에 의사를 어떻게 배치해야 가장 효과적인가?
 - 최대의 시청률을 얻기 위해 어떤 프로그램을 어느 시간대에 배치 해야하는가?



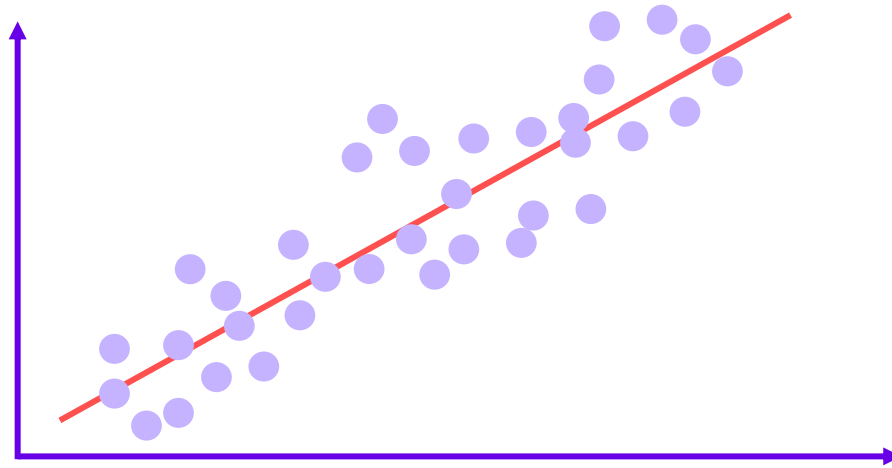
☑ 기계 학습 (Machine learning)

- 훈련 데이터로 학습된 특성을 활용하여 예측하는 기법
 - 넷플릭스 영화 추천 시스템
 - 유튜브 알고리즘



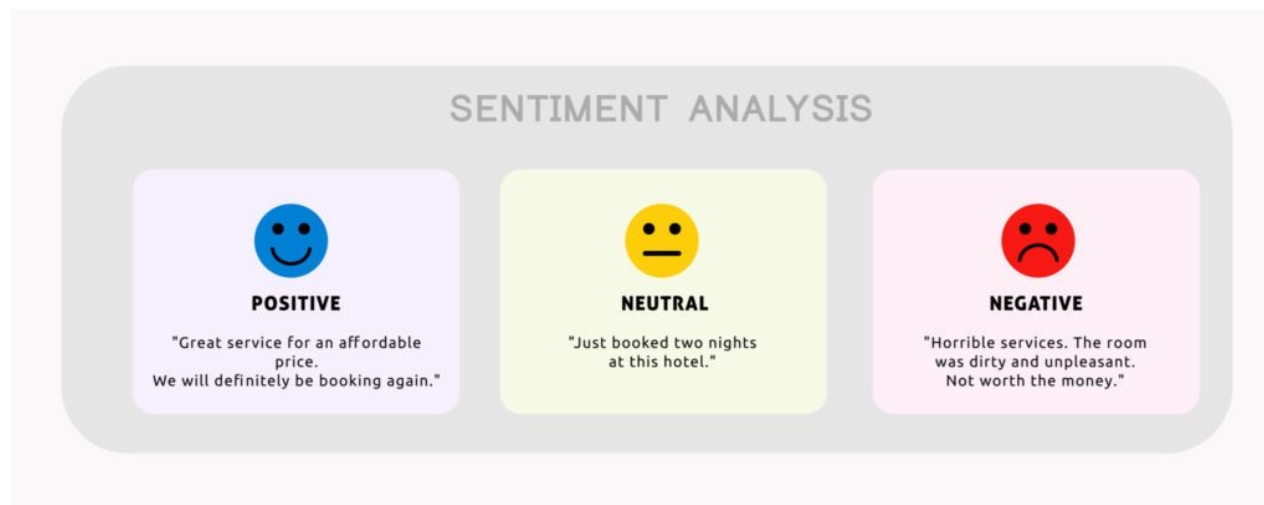
④ 회귀 분석 (Regression analysis)

- 독립 변수를 조작하며 종속 변수가 어떻게 변하는지를 확인하며 두 변인 관계를 파악하는 기법
 - 구매자의 나이와 구매 차량의 타입에 미치는 영향
 - 소비자의 만족도가 충성도에 미치는 영향



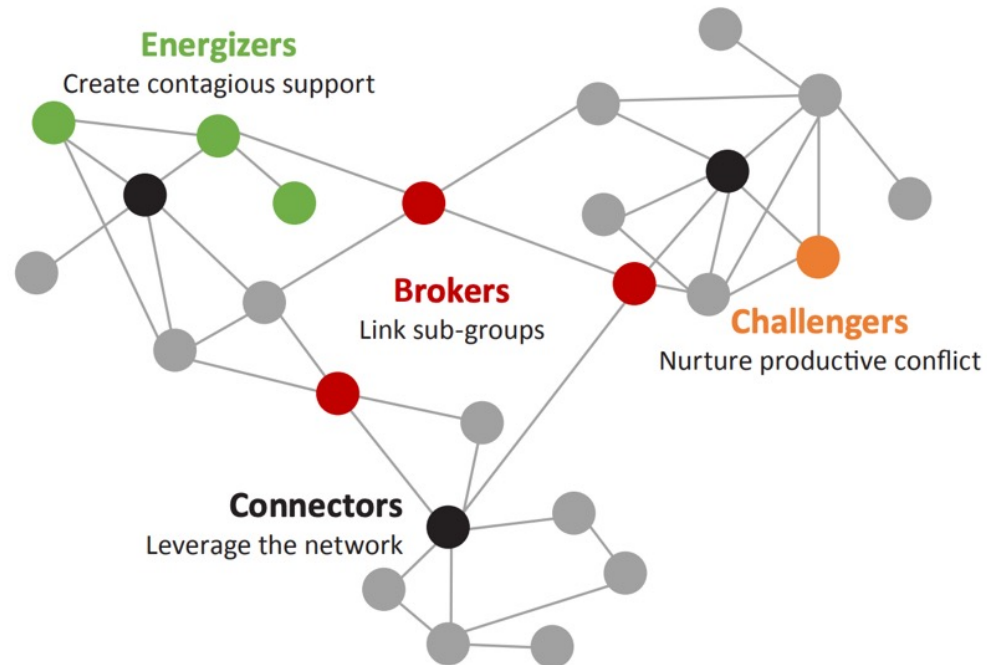
✓ 감정 분석 (Sentiment analysis)

- 자연어 데이터로부터 감정 시그널을 추출하는 기법
 - 신제품에 대한 고객의 평가
 - 뉴스 기사에 대한 댓글 분석



④ 소셜 네트워크 분석 (Social network analysis)

- 사회 관계망 분석
- 영향력 있는 사람 또는 고객 간 소셜커머스 관계를 파악할 수 있는 기법

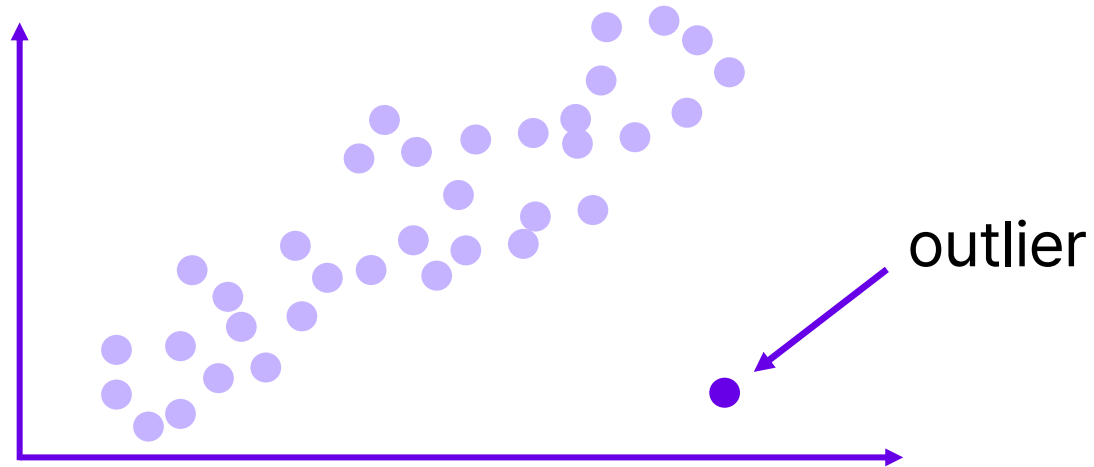


05

데이터 변환

☑ 이상치 (outlier)

- 관측되는 데이터의 범위에서 크게 벗어나는 값
- 결과적으로 잘못된 분석 결과를 초래할 수 있는 값



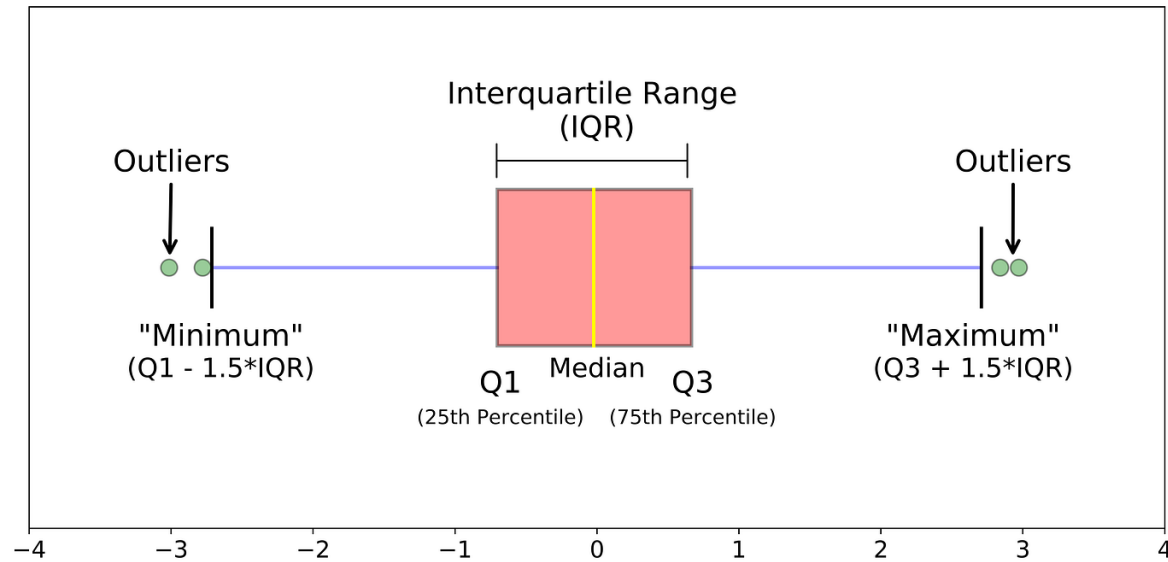
☑ 이상치가 데이터에 미치는 영향

Without Outlier	With Outlier
4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7	4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7, 300
Mean = 5.45	Mean = 30.00
Median = 5.00	Median = 5.50
Mode = 5.00	Mode = 5.00
Standard Deviation = 1.04	Standard Deviation = 85.03

적절한 이상치 처리가 필수적!

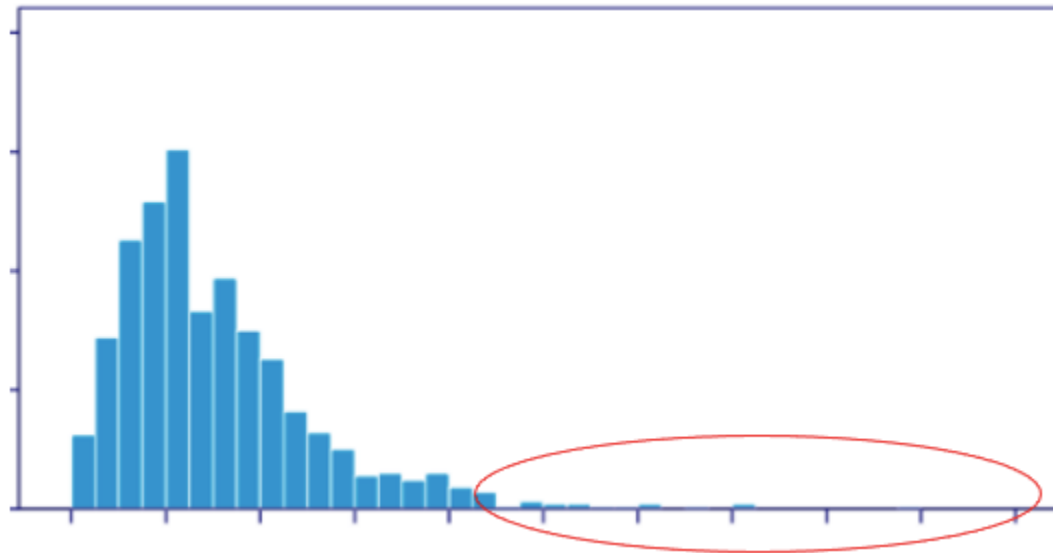
☑ 이상치 발견

1) Box-plot



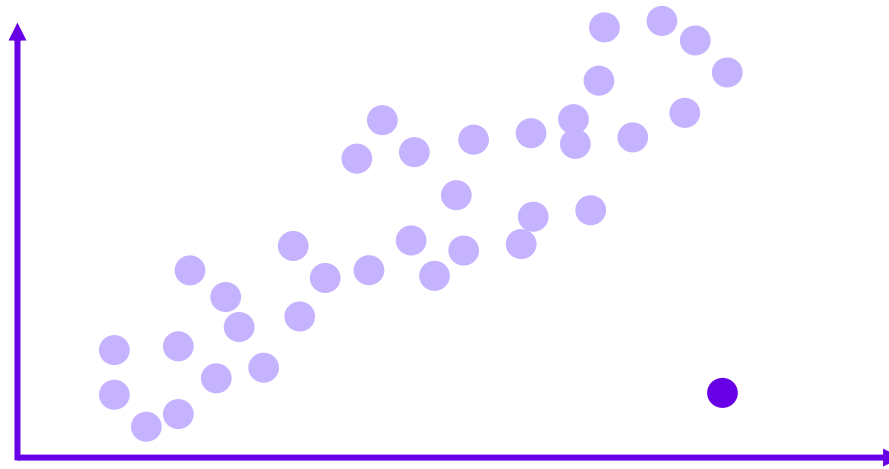
✓ 이상치 발견

2) Histogram



✓ 이상치 발견

3) Scatter Plot

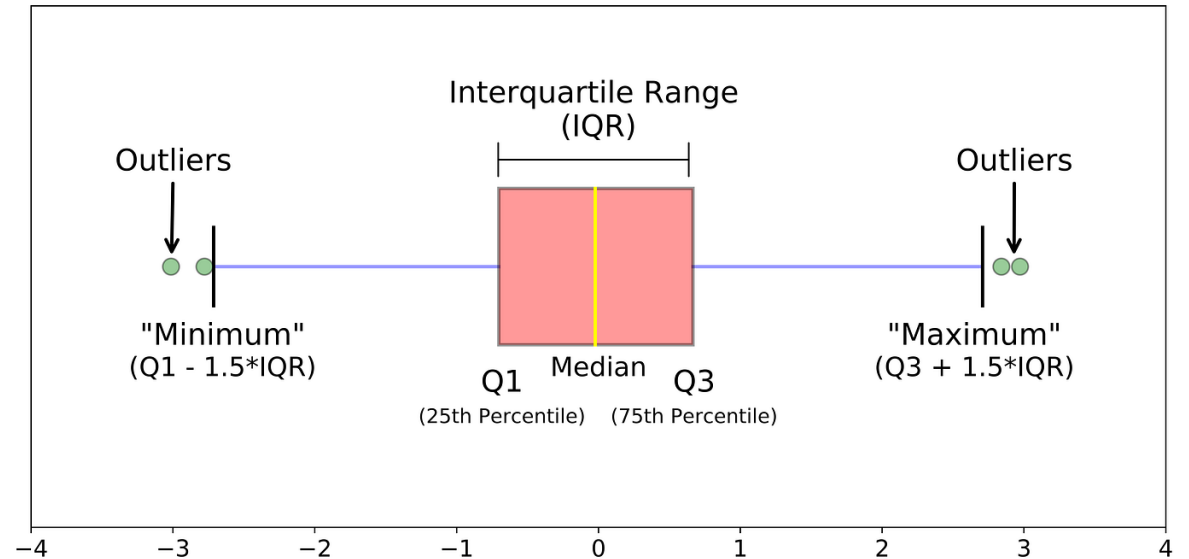


☑ 이상치 처리

1) 제거

2) 대체

- 평균값, 중앙값 등으로 대체
- 최댓값, 최솟값을 정하여 그 값으로 대체



✓ 결측치 (Missing Value)

- 값이 누락된 데이터

User	Device	OS	Transactions
A	Mobile	Android	5
B	Mobile	Android	3
C	NA	iOS	2
D	Tablet	Android	1
E	Mobile	iOS	4
F	NA	Android	2
G	Tablet	Android	4

Device	OS		Avg. Transactions
	#Android	#iOS	
Mobile	2	1	4
Tablet	2	0	2.5
Missing	2		

User	Device	OS	Transactions
A	Mobile	Android	5
B	Mobile	Android	3
C	Tablet	iOS	1
D	Tablet	Android	1
E	Mobile	iOS	4
F	Mobile	Android	2
G	Tablet	Android	4

Device	OS		Avg. Transactions
	#Android	#iOS	
Mobile	3	1	3.5
Tablet	2	1	2

✓ 결측치 처리

1) 제거

User	Device	OS	Transactions
A	Mobile	NA	5
B	Mobile	Android	3
C	NA	iOS	2
D	Tablet	Android	1
E	Mobile	iOS	4

2) 대체

- 평균값

OS	Revenue	OS	Global Mean	Group Mean
Android	1,804	Android	1,804	1,804
iOS	3,027	iOS	3,027	3,027
iOS	8,788	iOS	8,788	8,788
Android	NA	Android	4,145	2,696
Android	3,735	Android	3,735	3,735
Android	1,056	Android	1,056	1,056
iOS	9,319	iOS	9,319	9,319
Android	6,199	Android	6,199	6,199
Android	2,235	Android	2,235	2,235
iOS	NA	iOS	4,145	7,045
Android	1,146	Android	1,146	1,146

06

Pandas

👉 pandas란?

- 데이터 조작 및 분석을 위한 Python 프로그래밍 언어 용으로 작성된 소프트웨어 라이브러리
- 구조화된 데이터를 효과적으로 처리 및 저장 가능
- Numpy를 기반으로 설계됨

```
import pandas as pd
```



✓ Series 생성

- Series: 1차원 데이터를 저장하는 pandas의 기본 자료구조
- 하나의 Series 객체의 값은 동일한 데이터 타입의 값을 가짐

```
import pandas as pd

s = pd.Series([1, 3, 5, np.nan, 6, 8])
print(s)
```

Result	
0	1.0
1	3.0
2	5.0
3	NaN
4	6.0
5	8.0
dtype: float64	

✓ Series 생성

- 0~9까지 랜덤한 5개의 데이터로 Series 생성

```
import pandas as pd
import numpy as np

data = pd.Series(np.random.randint(10, size=5))
print(data)
```

Result

```
0    1
1    6
2    2
3    8
4    4
dtype: int64
```

07

Pandas 문법 (1)

✔ Series index 설정

```
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, size=5), index=["A", "B", "C", "D", "E"])
print(s)
```

Result

A	2
B	5
C	5
D	0
E	3

dtype: int64

👉 Series 내 index, value 값 확인

```
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, size=5), index=["A","B","C","D","E"])
print(s)
print()
print(s.index)
print(s.values)
print(s.A)
```

Result

```
A    2
B    8
C    3
D    0
E    2
```

```
dtype: int64
```

```
Index(['A', 'B', 'C', 'D', 'E'], dtype='object')
```

```
[2 8 3 0 2]
```

```
2
```

☑ indexing, slicing을 통한 Series 내 value 값 확인

```
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, size=5), index=["A", "B", "C", "D", "E"])
print(s)
print('-'*20)
print(s['C'])
print('-'*20)
print(s[['B', 'C', 'E']])
print('-'*20)
print(s[1::2])
```

Result

A	1
B	6
C	1
D	1
E	8

dtype: int64

1

B	6
C	1
E	8

dtype: int64

B	6
D	1

dtype: int64

✓ Series의 Broadcasting

```
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, size=5), index=["A", "B", "C", "D", "E"])
print(s)
print('-' * 10)
print(s * 10)
```

Result

A	2
B	0
C	0
D	5
E	5

dtype: int64

A	20
B	0
C	0
D	50
E	50

dtype: int64

④ 조건에 맞는 데이터 추출

```
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, size=5), index=["A", "B", "C", "D", "E"])
print(s)
print('-'*20)
print(s[s > 5])
```

Result

A	0
B	5
C	0
D	5
E	6

dtype: int64

E	6
---	---

dtype: int64

✔ Series와 dictionary

```
import pandas as pd

dic = {'D':3, 'F':7, 'E':5}
s = pd.Series(dic)
print(s)
print(type(dic))
print(type(s))
print()

for idx, val in s.items():
    print(idx, val)
```

Result

```
D    3
F    7
E    5
dtype: int64
<class 'dict'>
<class 'pandas.core.series.Series'>

D 3
F 7
E 5
```

☑ Series의 연산

- 연산 시 동일한 index에 대하여 연산됨
- index가 없는 경우에는 NaN 값이 됨

```
import pandas as pd

s1 = pd.Series([1, 2, 3, 4, 5], index=["A", "B", "C", "D", "E"])
s2 = pd.Series([10, 20, 30], index=["A", "C", "E"])

result = s1 + s2
print(s1)
print()
print(s2)
print()
print(result)
```

Result	
A	1
B	2
C	3
D	4
E	5
dtype: int64	
A	10
C	20
E	30
dtype: int64	
A	11.0
B	NaN
C	23.0
D	NaN
E	35.0
dtype: float64	

✓ Series 결측치 제거

- 결측치: 데이터에서 측정되지 않거나 누락된 값

```
import pandas as pd

s1 = pd.Series([1, 2, 3, 4, 5], index=["A", "B", "C", "D", "E"])
s2 = pd.Series([10, 20, 30], index=["A", "C", "E"])

result = s1 + s2
print(result)
print()
print(result.notnull())
print()
print(result[result.notnull()])
```

Result

```
A    11.0
B     NaN
C    23.0
D     NaN
E    35.0
dtype: float64
```

```
A     True
B    False
C     True
D    False
E     True
dtype: bool
```

```
A    11.0
C    23.0
E    35.0
dtype: float64
```

08

Pandas 문법 (2)

☑ DataFrame 생성

- DataFrame: Series의 집합으로 이루어진 데이터 타입

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21

```
import pandas as pd

df = pd.DataFrame(columns=["Email", "Name"])
df['Name'] = ['doori', 'minsu']
df['Email'] = ['dr@gmail.com', 'ms@gmail.com']
df['Age'] = [20, 21]

print(df)
print(type(df))
print(type(df['Name']))
```

Result			
	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
<class 'pandas.core.frame.DataFrame'>			
<class 'pandas.core.series.Series'>			

☑ dictionary로부터 DataFrame 생성

```
import pandas as pd

name = ["doori", "minsu"]
email = ["dr@gmail.com", "ms@gmail.com"]
dic = {"Name": name, "Email": email}
df = pd.DataFrame(dic)
print(df)
```

Result

	Name	Email
0	doori	dr@gmail.com
1	minsu	ms@gmail.com

✓ 인덱스 설정

```
import pandas as pd
```

```
df = pd.DataFrame(columns=["Email", "Name"], index=["one", "two"])
```

```
df["Name"] = ["doori", "minsu"]
```

```
df["Email"] = ["dr@gmail.com", "ms@gmail.com"]
```

```
df["Age"] = [20, 21]
```

```
print(df)
```

	Email	Name	Age
one	dr@gmail.com	doori	20
two	ms@gmail.com	minsu	21

Result

```
      Email  Name  Age
one  dr@gmail.com doori  20
two  ms@gmail.com minsu  21
```

☑ DataFrame의 요소 가져오기

```
df = pd.DataFrame(columns=["Email", "Name"], index=["one", "two"])
df["Name"] = ["doori", "minsu"]
df["Email"] = ["dr@gmail.com", "ms@gmail.com"]
df["Age"] = [20, 21]

print(df.index)
print(df.columns)
print(df.values)
```

	Email	Name	Age
one	dr@gmail.com	doori	20
two	ms@gmail.com	minsu	21

Result

```
Index(['one', 'two'], dtype='object')
Index(['Email', 'Name', 'Age'], dtype='object')
[['dr@gmail.com' 'doori' 20]
 ['ms@gmail.com' 'minsu' 21]]
```

④ 조건에 맞는 데이터 추출

```
import pandas as pd

df = pd.DataFrame(columns=["Email", "Name"])
df["Name"] = ["doori", "minsu"]
df["Email"] = ["dr@gmail.com", "ms@gmail.com"]
df["Age"] = [20, 21]

print(df[df['Name']=='doori'])
```

	Email	Name	Age
one	dr@gmail.com	doori	20
two	ms@gmail.com	minsu	21

Result

```
      Email  Name  Age
0  dr@gmail.com  doori   20
```

✓ iloc

- row index와 column index를 활용하여 데이터 접근
- DataFrame.loc[row_index, column_index]

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20

```
print(df.iloc[0, 1])
print()
print(df.iloc[[0, 3], 1])
print()
print(df.iloc[0:2, :2])
```

Result

```
doori

0    doori
3     hisu
Name: Name, dtype: object

      Email  Name
0  dr@gmail.com  doori
1  ms@gmail.com  minsu
```

✓ loc

- index name과 column name을 활용하여 데이터 접근
- DataFrame.loc[index_name, column_name]

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20

```
print(df.loc[0, "Name"])
print()
print(df.loc[[0, 3], "Name"])
print()
print(df.loc[0:2, : "Name"])
```

Result

doori

0 doori

3 hisu

Name: Name, dtype: object

	Email	Name
0	dr@gmail.com	doori
1	ms@gmail.com	minsu
2	hn@gmail.com	hana

☑ row(행) 추가

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy#@gmail.com	hyun	20



	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20
5	boo@gmail.com	boo	21

```
df.loc[5] = {"Email": "boo@gmail.com", "Name": "boo", "Age": 21}  
print(df)
```

Result			
	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20
5	boo@gmail.com	boo	21

✓ column(열) 추가

- 리스트를 이용하여 추가

```
df["Address"] = ["Seoul", "Busan", "Jeju", "Deagu", "Seoul"]  
print(df)
```

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20

Result				
	Email	Name	Age	Address
0	dr@gmail.com	doori	20	Seoul
1	ms@gmail.com	minsu	21	Busan
2	hn@gmail.com	hana	25	Jeju
3	hs@gmail.com	hisu	26	Deagu
4	hy@gmail.com	hyun	20	Seoul

✓ shape

- DataFrame의 행, 열 개수를 tuple 형태로 반환

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20

```
print(df.shape)
```

Result

(5, 3)

✓ concat

- concat을 이용하여 서로 다른 DataFrame을 합칠 수 있음

```
import pandas as pd

name1 = ["doori", "minsu", "hisu"]
age1 = [20, 21, 26]
name2 = ["hana", "hyun", "boo"]
age2 = [21, 25, 20]
dic1 = {"Name": name1, "Age": age1}
dic2 = {"Name": name2, "Age": age2}

df1 = pd.DataFrame(dic1)
df2 = pd.DataFrame(dic2)
print(df1)
print('-'*15)
print(df2)
print('-'*15)
df3 = pd.concat([df1, df2]).reset_index(drop=True)
print(df3)
```

Result

	Name	Age
0	doori	20
1	minsu	21
2	hisu	26

	Name	Age
0	hana	21
1	hyun	25
2	boo	20

	Name	Age
0	doori	20
1	minsu	21
2	hisu	26
3	hana	21
4	hyun	25
5	boo	20

✓ concat

- concat을 이용하여 서로 다른 DataFrame을 합칠 수 있음

```
df3 = pd.concat([df1, df2], axis=1)  
print(df3)
```

Result				
	Name	Age	Name	Age
0	doori	20	hana	21
1	minsu	21	hyun	25
2	hisu	26	boo	20

☑ Transpose (전치)

- 행과 열을 바꾸어 줌

```
print(df)
print()
print(df.T)
```

Result

```

      Email  Name  Age
0  dr@gmail.com doori  20
1  ms@gmail.com minsu  21
2  hn@gmail.com hana  25
3  hs@gmail.com hisu  26
4  hy@gmail.com hyun  20

      0      1      2      3      4
Email dr@gmail.com ms@gmail.com hn@gmail.com hs@gmail.com hy@gmail.com
Name   doori      minsu      hana      hisu      hyun
Age      20      21      25      26      20
```

	Email	Name	Age
0	dr@gmail.com	doori	20
1	ms@gmail.com	minsu	21
2	hn@gmail.com	hana	25
3	hs@gmail.com	hisu	26
4	hy@gmail.com	hyun	20



	0	1	2	3	4
Email	dr@gmai l.com	ms@gm ail.com	hn@gma il.com	hs@gma il.com	hy@gma il.com
Name	doori	minsu	hana	hisu	hyun
Age	20	21	25	26	20

09

데이터 집계 및 요약

☑ 데이터 프레임 생성

```
import pandas as pd

names = ["apple", "pear", "banana", "strawberry", "mango", "cherry", "melon"]
quantity = [3, 5, 1, None, 1, 5, 5]
fruit_df = pd.DataFrame({'Name': names, 'Quantity': quantity})
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

④ 기초 통계 (sum, mean, var, std)

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

```
print('Quantity sum:', fruit_df.Quantity.sum())
print('Quantity mean:', fruit_df.Quantity.mean())
print('Quantity var:', fruit_df.Quantity.var())
print('Quantity std:', fruit_df.Quantity.std())
```

Result

```
Quantity sum: 20.0
Quantity mean: 3.3333333333333335
Quantity var: 3.8666666666666667
Quantity std: 1.9663841605003503
```

④ 기초 통계 (min, max, argmin, argmax)

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

```
print('Quantity min:', fruit_df.Quantity.min())  
print('Quantity max:', fruit_df.Quantity.max())  
print('Quantity argmin:', fruit_df.Quantity.argmin())  
print('Quantity argmax:', fruit_df.Quantity.argmax())
```

Result

```
Quantity min: 1.0  
Quantity max: 5.0  
Quantity argmin: 2  
Quantity argmax: 1
```

✓ describe()

- 통계치 요약

```
print(fruit_df.describe())
```

Result

	Quantity
count	6.000000
mean	3.333333
std	1.966384
min	1.000000
25%	1.500000
50%	4.000000
75%	5.000000
max	5.000000

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

✔ sort_values()

```
print(fruit_df.sort_values(by='Quantity'))  
print()  
print(fruit_df.sort_values(by='Name', ascending=False))  
print()
```

Result		
	Name	Quantity
2	banana	1.0
4	mango	1.0
0	apple	3.0
1	pear	5.0
5	cherry	5.0
6	melon	5.0
3	strawberry	NaN
	Name	Quantity
3	strawberry	NaN
1	pear	5.0
6	melon	5.0
4	mango	1.0
5	cherry	5.0
2	banana	1.0
0	apple	3.0

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

✓ rank()

```
fruit_df['Rank'] = fruit_df.Quantity.rank()
print(fruit_df)
print()
fruit_df['Rank'] = fruit_df.Quantity.rank(ascending=False)
print(fruit_df)
```

Result			
	Name	Quantity	Rank
0	apple	3.0	3.0
1	pear	5.0	5.0
2	banana	1.0	1.5
3	strawberry	NaN	NaN
4	mango	1.0	1.5
5	cherry	5.0	5.0
6	melon	5.0	5.0

	Name	Quantity	Rank
0	apple	3.0	4.0
1	pear	5.0	2.0
2	banana	1.0	5.5
3	strawberry	NaN	NaN
4	mango	1.0	5.5
5	cherry	5.0	2.0
6	melon	5.0	2.0

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

✔ counts

```
print(fruit_df.count())
print()
print(fruit_df['Quantity'].value_counts())
print()
print(fruit_df['Name'].value_counts())
```

Result	
Name	7
Quantity	6
dtype: int64	
5.0	3
1.0	2
3.0	1
Name: Quantity, dtype: int64	
apple	1
pear	1
banana	1
strawberry	1
mango	1
cherry	1
melon	1
Name: Name, dtype: int64	

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

✓ 원소 체크

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

```
if 'apple' in list(fruit_df['Name']):  
    print('True')
```

Result

True

✓ info()

```
print(fruit_df.info())
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

Result

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 7 entries, 0 to 6  
Data columns (total 2 columns):  
  
#   Column      Non-Null Count  Dtype  
---  -  
0   Name        7 non-null      object  
1   Quantity    6 non-null      float64  
dtypes: float64(1), object(1)  
memory usage: 240.0+ bytes  
None
```

☑ head()

```
print(fruit_df.head()) #기본 5개 출력  
print()  
print(fruit_df.head(2)) #개수 지정 가능
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

Result		
	Name	Quantity
0	apple	3.0
1	pear	5.0
2	banana	1.0
3	strawberry	NaN
4	mango	1.0
	Name	Quantity
0	apple	3.0
1	pear	5.0

✓ tail()

```
print(fruit_df.tail()) #기본 5개 출력  
print()  
print(fruit_df.tail(2)) #개수 지정 가능
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

Result

	Name	Quantity	Rank
2	banana	1.0	5.5
3	strawberry	NaN	NaN
4	mango	1.0	5.5
5	cherry	5.0	2.0
6	melon	5.0	2.0

	Name	Quantity	Rank
5	cherry	5.0	2.0
6	melon	5.0	2.0

✔ isnull()

```
print(fruit_df.isnull())
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

Result		
	Name	Quantity
0	False	False
1	False	False
2	False	False
3	False	True
4	False	False
5	False	False
6	False	False

✔ isnull()

```
print(fruit_df.isnull().sum())
```

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

Result

```
Name      0  
Quantity   1  
dtype: int64
```

✓ unique()

- 개별 컬럼의 유일값 출력

	Name	Quantity
0	apple	3
1	pear	5
2	banana	1
3	strawberry	nan
4	mango	1
5	cherry	5
6	melon	5

```
print(fruit_df['Quantity'].unique())  
print(fruit_df['Name'].unique())
```

Result

```
[ 3.  5.  1. nan]  
['apple' 'pear' 'banana' 'strawberry' 'mango' 'cherry' 'melon']
```

👉 csv/excel 파일 읽기, 쓰기

```
fruit_df.to_csv('./fruit.csv', index=False)
df = pd.read_csv('./fruit.csv')
print(df)
print()
fruit_df.to_excel('./fruit.xlsx', sheet_name='Sheet1', index=False)
df = pd.read_excel('./fruit.xlsx', sheet_name='Sheet1')
print(df)
```

	A	B	C
1	Name	Quantity	
2	apple	3	
3	pear	5	
4	banana	1	
5	strawberry		
6	mango	1	
7	cherry	5	
8	melon	5	

fruit.csv

	A	B	
1	Name	Quantity	
2	apple	3	
3	pear	5	
4	banana	1	
5	strawberry		
6	mango	1	
7	cherry	5	
8	melon	5	

fruit.xlsx

Result

	Name	Quantity
0	apple	3.0
1	pear	5.0
2	banana	1.0
3	strawberry	NaN
4	mango	1.0
5	cherry	5.0
6	melon	5.0

	Name	Quantity
0	apple	3.0
1	pear	5.0
2	banana	1.0
3	strawberry	NaN
4	mango	1.0
5	cherry	5.0
6	melon	5.0